

21世纪软件工程专业规划教材

软件工程导论（第6版）

第4章 总体设计

- 总体设计的基本目的就是回答“**概括**地说，系统应该**如何实现**”这个问题，因此，总体设计又称为概要设计或初步设计。
- 总体设计阶段的另一项重要任务是**设计软件的结构**，也就是要确定系统中每个程序是由哪些模块组成的，以及这些模块相互间的关系。

目录

CONTENTS

- 1 设计过程
- 2 设计原理
- 3 启发规则
- 4 描绘软件结构的图形工具
- 5 面向数据流的设计方法

设计过程

总体设计通常由**系统设计阶段**和**结构设计阶段**组成。系统设计阶段确定系统的具体实现方案；结构设计阶段确定软件结构。

系统设计阶段：

设想供选择的方案：利用数据流图中的分组方案，抛弃技术上行不通的方案，得到可能的实现策略。

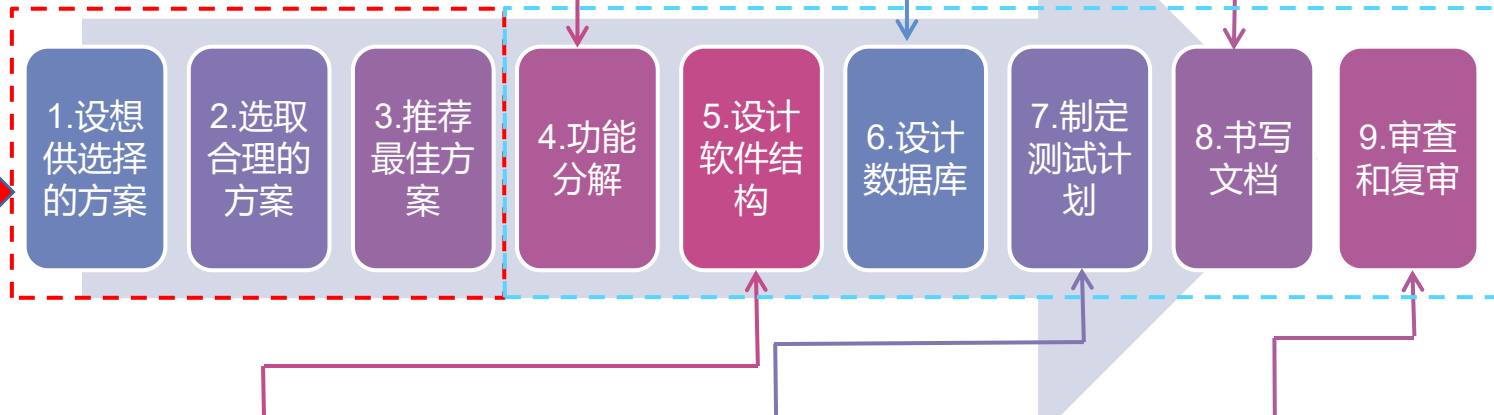
选取合理的方案：在可行方案中选取低成本、中等成本及高成本3种方案，结合用户意见判断哪些更合理。

推荐最佳方案：分析员综合分析对比各种方案的利弊，推荐出一个最佳方案。

对程序（特别是复杂的大型程序）的设计，通常分为两个阶段完成：首先进行**结构设计**，然后进行**过程设计**。为确定软件结构，需要从实现角度把复杂的功能进一步分解成一系列比较简单的功能。

对于需要使用数据库的那些应用系统，软件工程师应该在需求分析阶段所确定的系统数据需求的基础上，进一步设计**数据库**。

应该**用正式的文档**记录总体设计的结果，在这个阶段应该完成的文档通常有下述几种：系统说明、用户手册、测试计划、详细的实现计划、数据库设计结果



通常程序中的一个模块完成一个适当的子功能。应该把模块组织成良好的**层次系统**，顶层模块调用它的下层模块以实现程序的完整功能，每个下层模块再调用更下层的模块，完成程序的一个子功能，**最下层的模块完成最具体的功能**。

在软件开发的早期阶段考虑**测试问题**，能促使软件设计人员在设计时注意提高软件的可测试性。

最后应该对总体设计的结果进行严格的**技术审查**，在技术审查通过之后再由客户从管理角度进行**复审**。

设计原理

模块化是一种将系统或产品分解为相互独立、可以独立设计、开发、测试和维护的**模块**（组件、服务等）的过程。

在软件开发中，模块化设计将一个**大型软件系统**划分为多个独立的模块，每个模块完成一个子功能，并通过**接口**与其他模块进行通信。这种设计模式使软件**易开发、易调试、易测试**，其意义在于最大化**重用设计**，以最少的模块及零部件满足更多的个性化需求。

模块化是为了使一个复杂的大型程序能被人的智力所管理，**模块是构成程序的基本构件。**

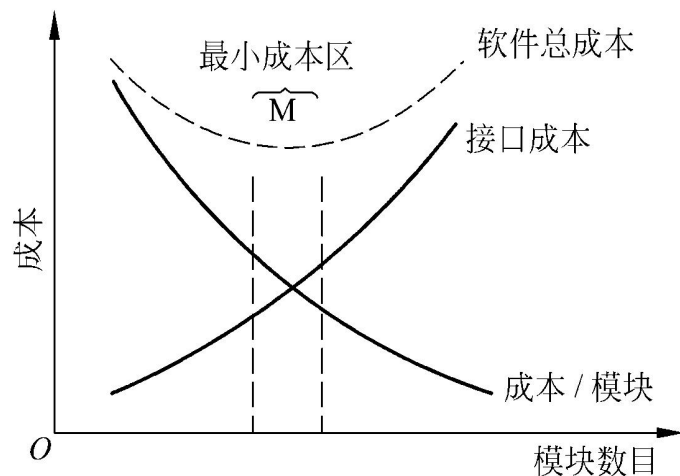
函数 $C(x)$ 定义问题 x 的**复杂程度**，函数 $E(x)$ 确定解决问题 x 需要的**工作量**（时间）。对于两个问题 $P1$ 和 $P2$ ，如果 $C(P1) > C(P2)$ ，则 $E(P1) > E(P2)$ 。

根据人类解决一般问题的经验，另一个有趣的规律是

$$C(P1 + P2) > C(P1) + C(P2)$$

$$E(P1 + P2) > E(P1) + E(P2)$$

这个不等式导致“**各个击破**”的结论——把复杂的问题分解成许多容易解决的小问题，原来的问题也就容易解决了。这就是模块化的根据。



然而**无限分割软件并不合适**，当模块数目增加时每个模块的规模将减小，开发单个模块需要的成本（工作量）确实减少了；但是，随着模块数目增加，设计**模块间接口**所需要的工作量也将增加。

根据这两个因素，得出了图中的总成本曲线。每个程序都相应地有一个最适当的模块数目 M ，使得系统的**开发成本最小**。

抽象

在总体设计中，抽象是一种忽略不必要的细节，专注于本质特征和契约的思维方法。其目的：管理复杂性，隔离变化，建立清晰的边界。

“忽略什么”比“关注什么”更重要。通过隐藏“如何做”的细节，只暴露“做什么”的契约。

业务概念 抽象	将混乱的现实世界业务规则、流程和实体，抽象为领域模型、业务流程和用例。
系统架构 抽象	将整个系统抽象为一些高层次组件及其关系，不关心内部实现
组件/模块 抽象	定义每个主要组件对外提供的服务接口，隐藏其内部复杂的类、算法和数据。
技术基础设施 抽象	对底层技术细节进行统一封装，提供更简洁、稳定的编程接口。

抽象在设计中主要通过“定义接口，隐藏实现”来体现
接口是抽象的具体化身

逐步求精

从对问题的高度概括、抽象的表述开始，然后通过连续的精化步骤，逐步增加细节，直到获得一个可以用编程语言直接实现的、足够具体的设计描述。



逐步求精最初是由瑞士计算机科学家尼古拉斯·威茨(Niklaus Wirth)提出的一种自顶向下的设计策略。按照这种设计策略，程序的体系结构是通过逐步精化处理过程的层次而设计出来的。通过逐步分解对功能的宏观陈述而开发出层次结构，直至最终得出用程序设计语言表达的程序。

- “自顶向下，逐步求精”是结构化程序设计的常见思路
- 用经典的“订单处理系统”为例来展示逐步求精的过程：
- ① 第1级求精：系统上下文与最高级目标：“构建一个在线订单处理系统。”“系统包含用户界面、订单核心服务、外部适配器。”
 - ② 第2级求精：分解为子系统/高层次模块：细化核心服务，定义高级数据流；
 - ③ 第3级求精：定义子系统接口与职责：“支付处理子系统负责处理支付。”
 - ④ 第4级求精：子系统内部架构设计：“支付处理子系统内部如何工作？”
 - ⑤ 第N级求精：直至可编码实现：最终，每一个类的方法逻辑、算法细节、异常处理、日志格式等，都被清晰地定义出来，可以直接交给开发人员编写代码。

假设我们有一个支付模块。

1. 应用信息隐藏:

- 我们定义一个 `IPaymentGateway` 接口, 包含 `charge(amount, cardInfo)` 方法。
- 系统其他部分 (如订单服务) 只依赖这个接口。
- 在接口背后, 我们隐藏了:
 - 实际调用的是支付宝还是Stripe的SDK。
 - 内部的重试逻辑、加密算法。
 - 记录审计日志的具体格式和存储位置。

2. 应用局部化:

- 我们将所有与“支付”相关的代码放在一个物理模块或项目 `payment-service` 中。
- 这个模块内包含了:
 - `IPaymentGateway` 接口及其多个实现 (`AlipayGateway`, `StripeGateway`) 。
 - 支付领域模型 (`Payment`, `Transaction`) 。
 - 支付相关的配置、错误码、工具类。
 - 支付流程的状态机实现。
- 所有与支付相关的知识都高度集中于此。

够低成本应对变化的系统。

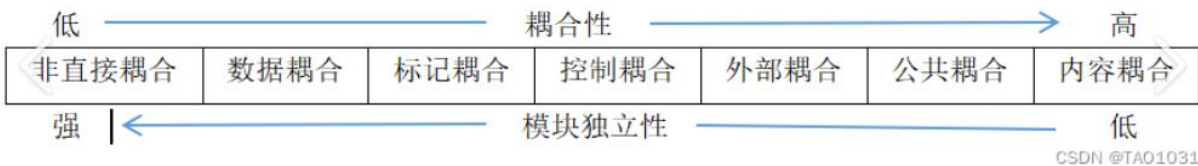
模块独立

模块独立: 是指系统中的每个模块 (可以是类、包、组件、微服务) 应具有明确的单一职责, 并且与其他模块的关联尽可能少且简单。它包含两个相辅相成、必须同时追求的维度:

高内聚: 模块内部各元素彼此结合的紧密程度高。

低耦合: 模块之间相互依赖的关联程度低。

耦合是对一个软件结构内不同模块之间**互连程度的度量**。耦合程度强烈影响着系统的可理解性、可测试性、可靠性和可维护性。



非直接耦合 两个模块之间没有任何依赖关系 (**最理想**)

数据耦合: 两个模块之间通过参数 (通常是必要的) 进行交互, 传递的是基础数据类型, 而不是复杂的对象 (**良好**)

标记耦合: 模块之间共享一个复合数据结构 (如结构体或类), 但可能只用到了数据结构中的部分数据 (**不必要的依赖**)

控制耦合: 一个模块传递信息给另一个模块, 用来控制后者的逻辑流程 (**需谨慎**)

外部耦合: 模块依赖于同一个外部协议、格式或设备。有时不可避免, 但应将其封装。

公共耦合: 当多个模块共享同一个全局数据时发生的耦合 (**非常危险**)

内容耦合: 一个模块直接使用或修改另一个模块的内部数据或逻辑。 (**绝对禁止**)

内聚：内聚衡量一个模块**内部各个元素**彼此结合的**紧密程度**。

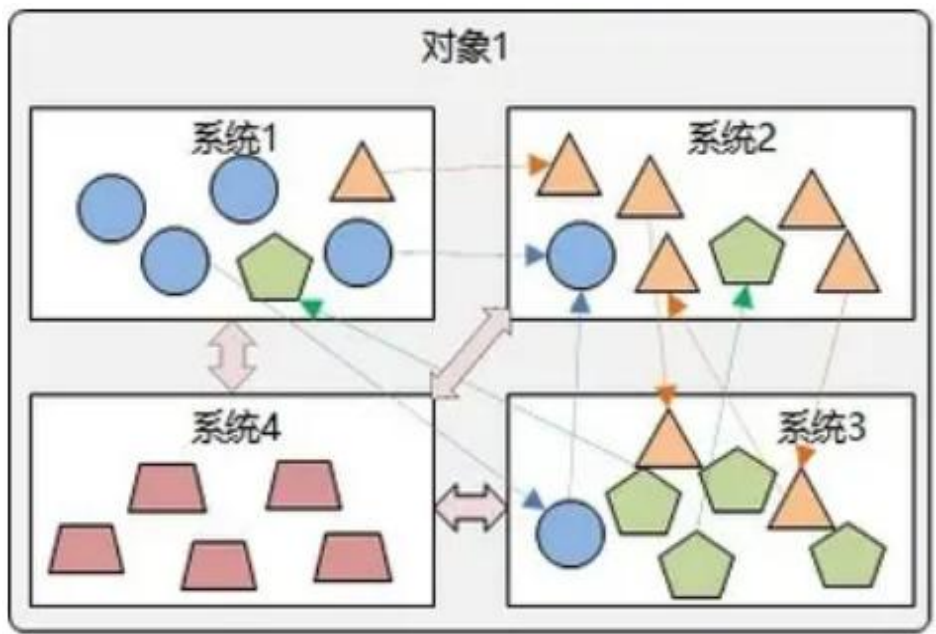
它是信息隐藏和局部化概念的自然扩展。模块内部的元素（数据、函数、类）共同完成一个紧密相关的单一功能或一组强相关功能。

内聚和耦合是密切相关的，模块内的高内聚往往意味着模块间的松耦合。

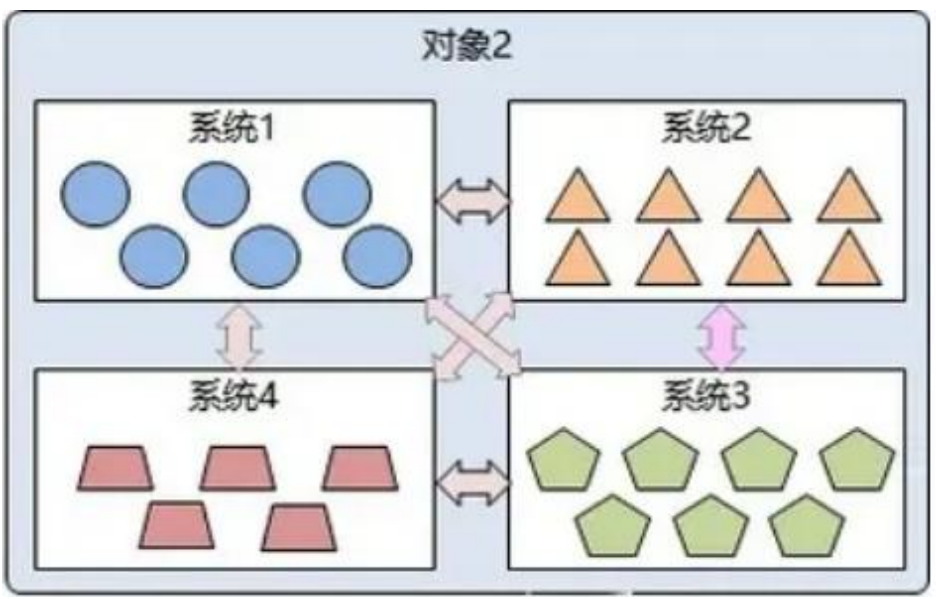
内聚分为三大类**低内聚**、**中内聚**和**高内聚**。



低内聚	偶然内聚：模块内的元素之间毫无关系，只是被偶然地放在一起（最差）
	逻辑内聚：一个模块完成的任务在逻辑上属于相同或相似的一类（较差）
	时间内聚：一个模块包含的任务必须在同一段时间内执行（不理想）
中内聚	过程内聚：一个模块内的处理元素是相关的，而且必须以特定次序执行（一般）
	通信内聚：所有元素都使用同一个输入数据和（或）产生同一个输出数据（较好）
高内聚	顺序内聚：一个模块内的处理元素和同一个功能密切相关，而且这些处理必须顺序执行（通常一个处理元素的输出数据作为下一个处理元素的输入数据）（好）
	功能内聚：模块内所有处理元素属于一个整体，完成一个单一的功能（最佳）



低内聚，高耦合的状态

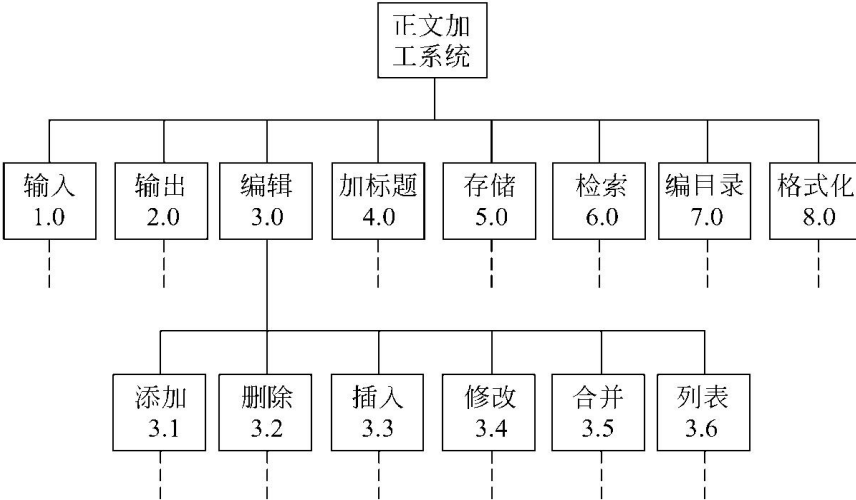
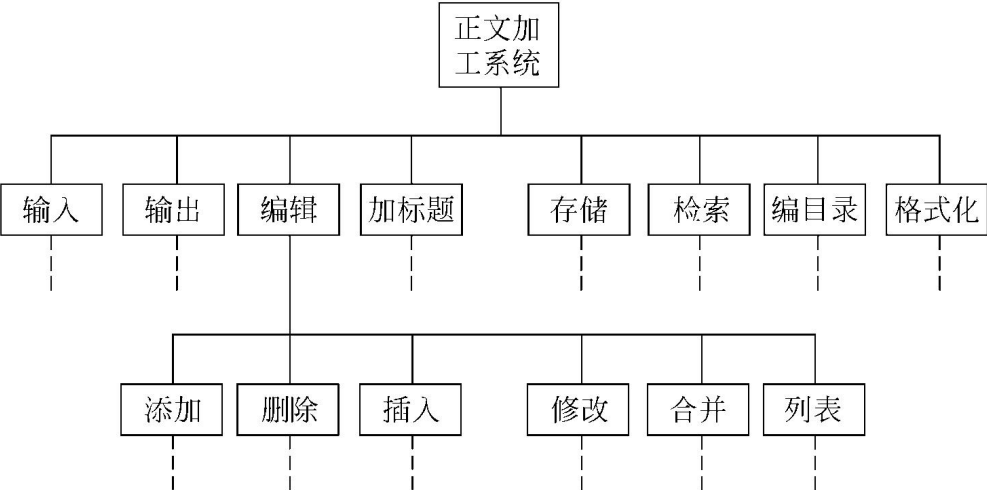


高内聚，低耦合的状态

描绘软件结构的图形工具

层次图是描述系统模块层次结构的静态视图。用来描绘软件的**层次结构**。

数据结构的层次方框图相同，但是表现的内容却完全不同（层次图中的一个方框代表一个模块，方框间的连线代表调用关系，层次方框图中的连线代表组成关系）。层次图很适于在**自顶向下**设计软件的过程中使用。



HIPO图是美国IBM公司发明的“层次图加输入/处理/输出图”的英文缩写。为了能使HIPO图具有可追踪性，在H图（层次图）里除了最顶层的方框之外，每个方框都加了编号。模块在H图中的编号，可以追踪了解该模块在系统中的位置

IPO表

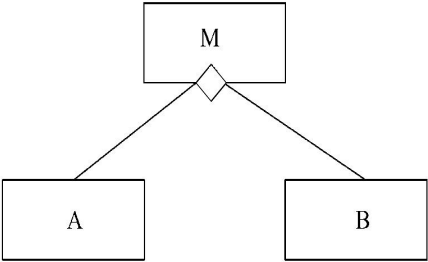
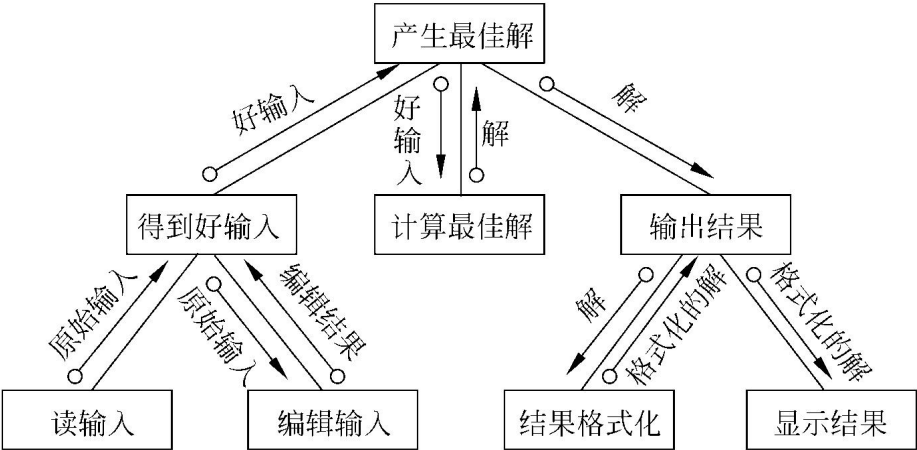
系统：图书管理系统
模块：借书处理
编号：3.1
所属层次：图书馆管理系统 -> 借阅管理 -> 借书

输入：模块：借书处理
1. 读者ID（从读卡器或界面输入）
2. 图书ISBN/条形码（从扫描器输入）
3. 当前系统日期

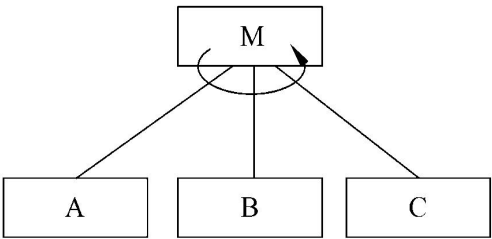
输出：
1. 数据库更新（借阅记录、图书、读者）
2. 屏幕提示信息（成功/失败）
3. 可能的打印凭条（借书小票）

处理：
1. 验证读者状态（是否有效、有无超期、未缴罚款）
2. 验证图书状态（是否在馆、是否可借）
3. IF 所有验证通过 THEN
在借阅记录表中插入新记录
更新图书状态为“已借出”
更新读者已借数量
生成借书成功提示
ELSE
生成具体错误提示（如“读者已挂失”）
END IF

结构图是一种用于描述软件系统模块**层次结构**和**调用关系**的图形化工具。图中一个方框代表一个模块，框内注明模块的名字或主要功能；方框之间的箭头（或直线）表示模块的调用关系。尾部是空心圆表示传递的是数据，实心圆表示传递的是控制信息。

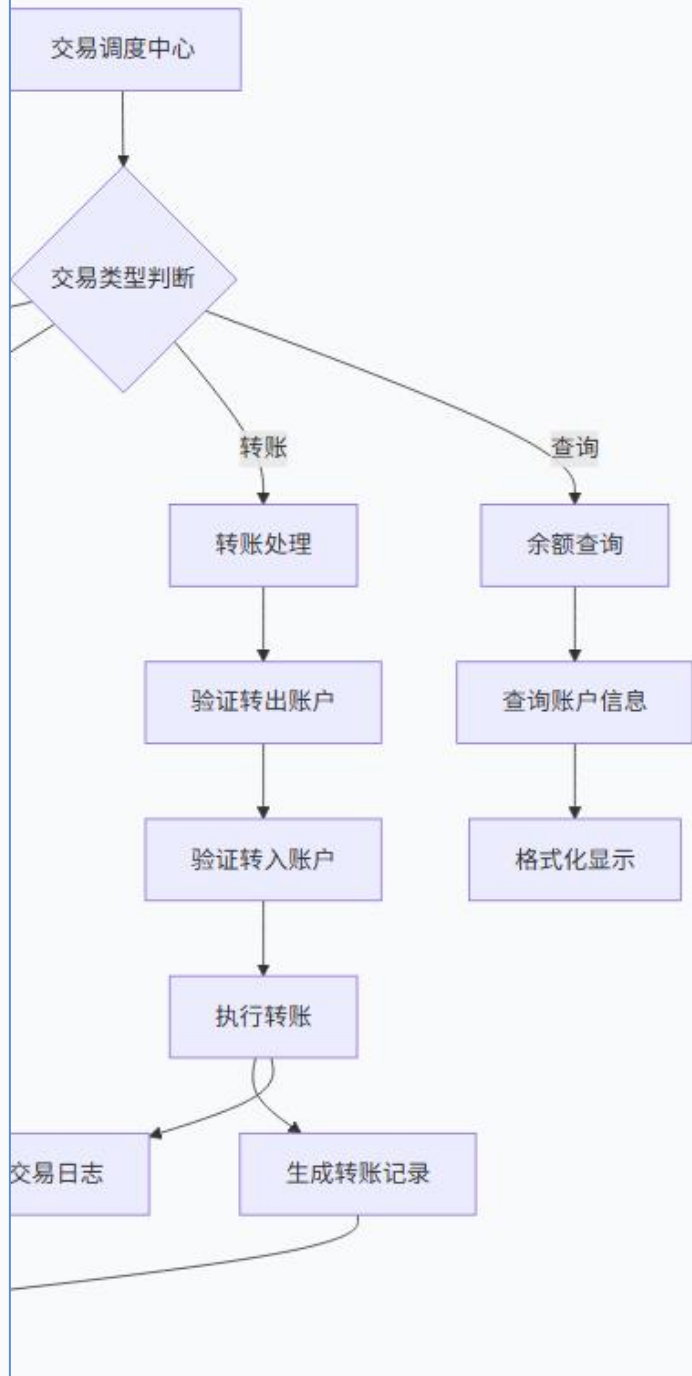
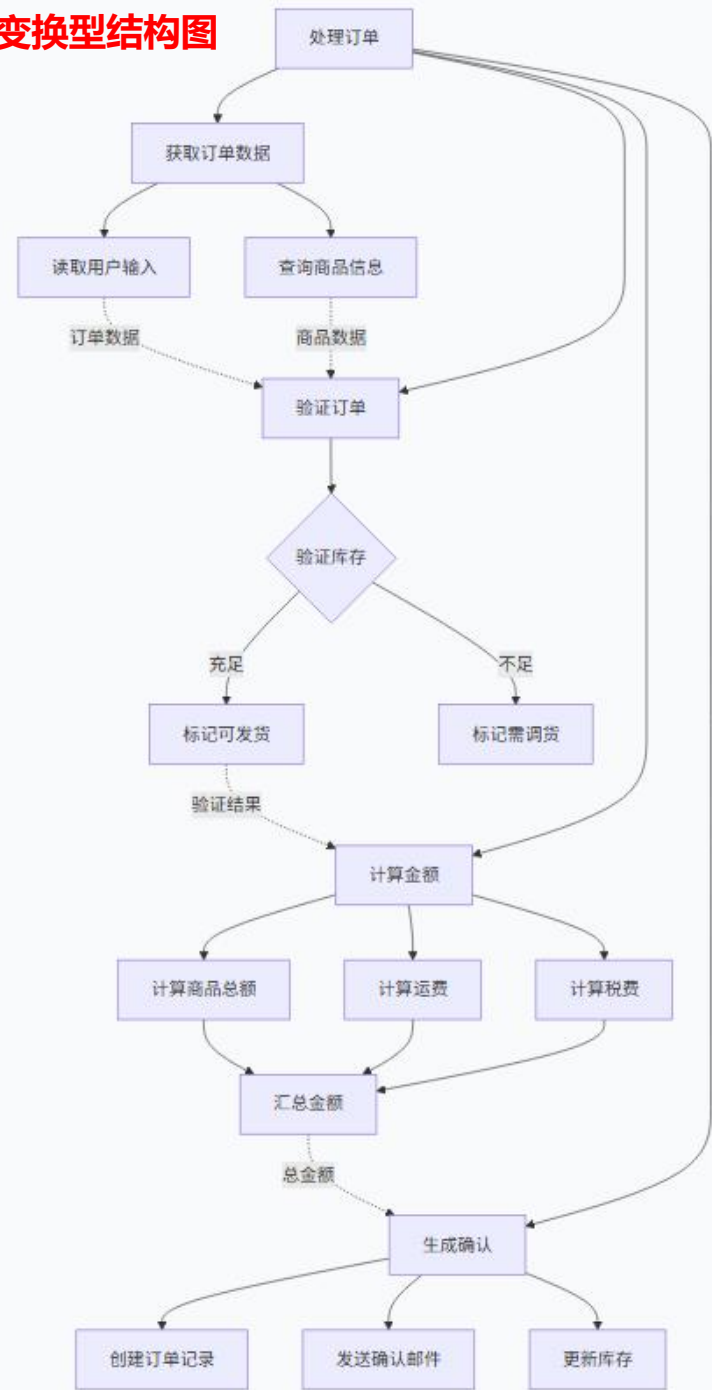


判定为真时调用A，为假时调用B

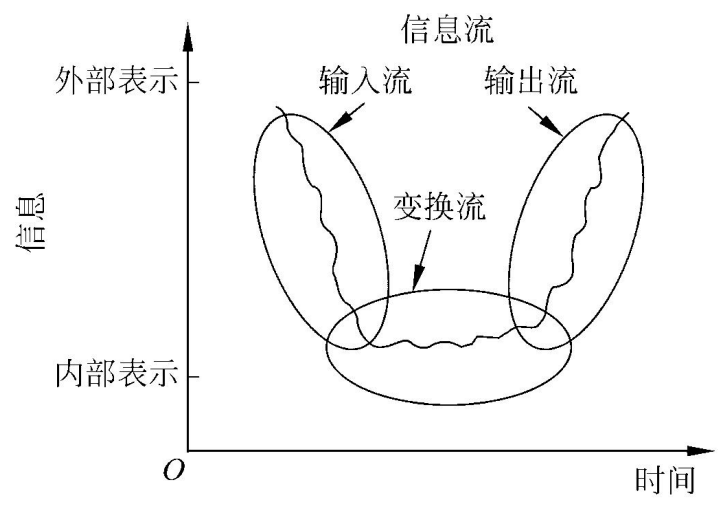


模块M循环调用A,B,C

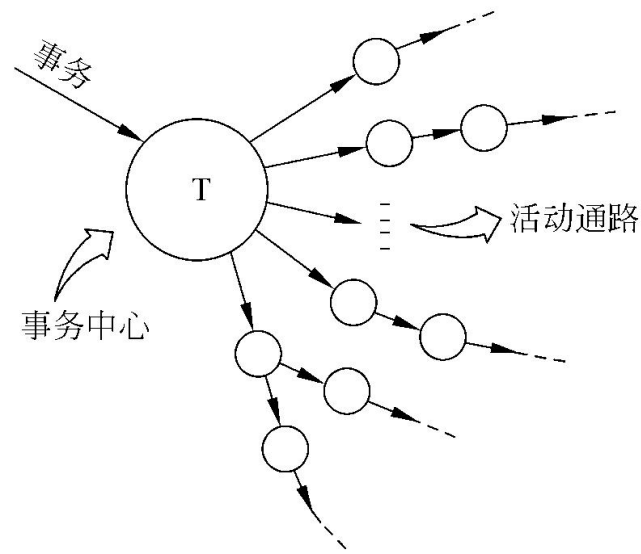
变换型结构图



输入系统，由外部
系统的信息通过
再沿输出通路变
。当数据流图具
就叫作变换流。

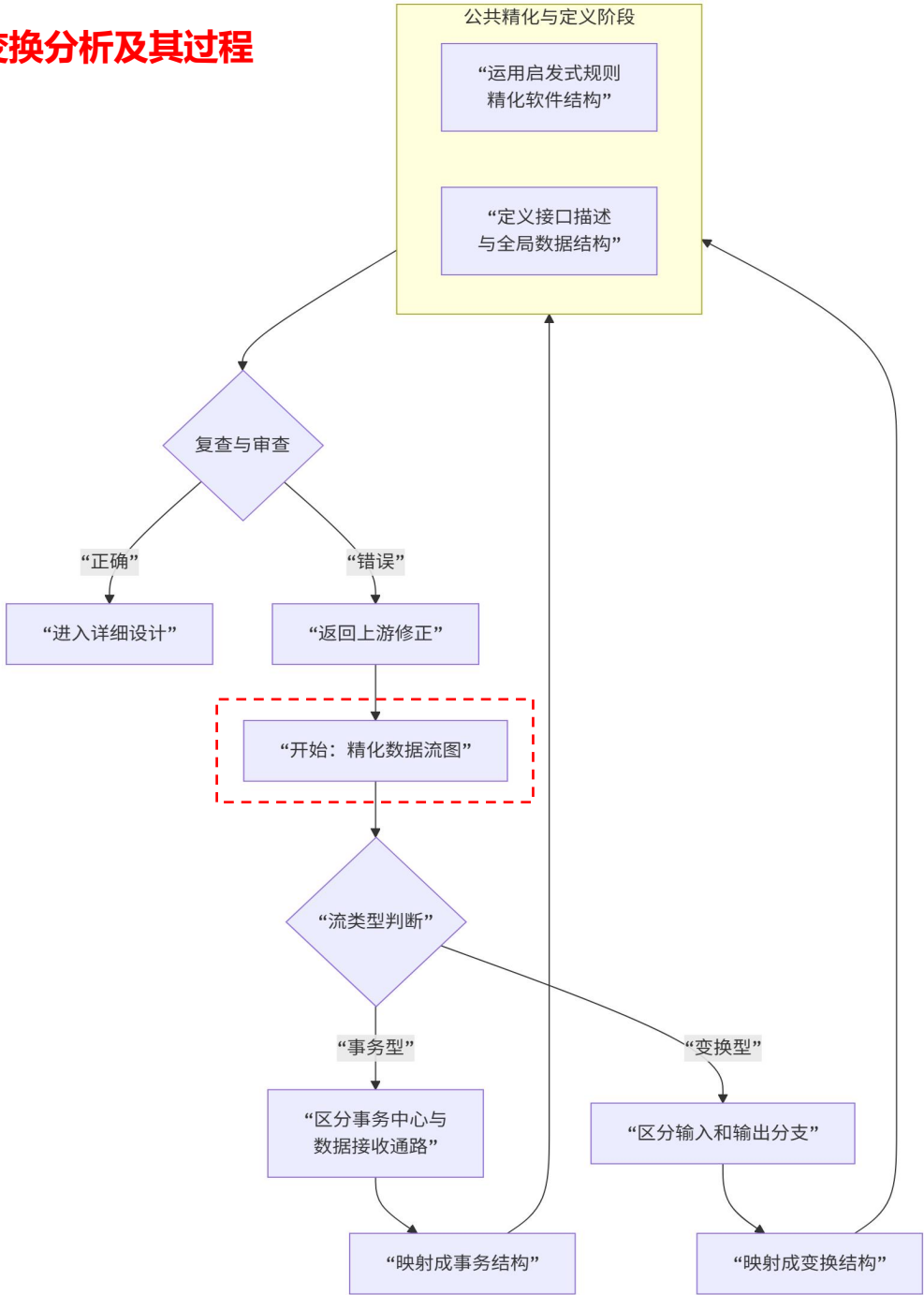


到达一个处理T，
型在若干个动作
类数据流应该划
事务流。图中的
成下述任务：

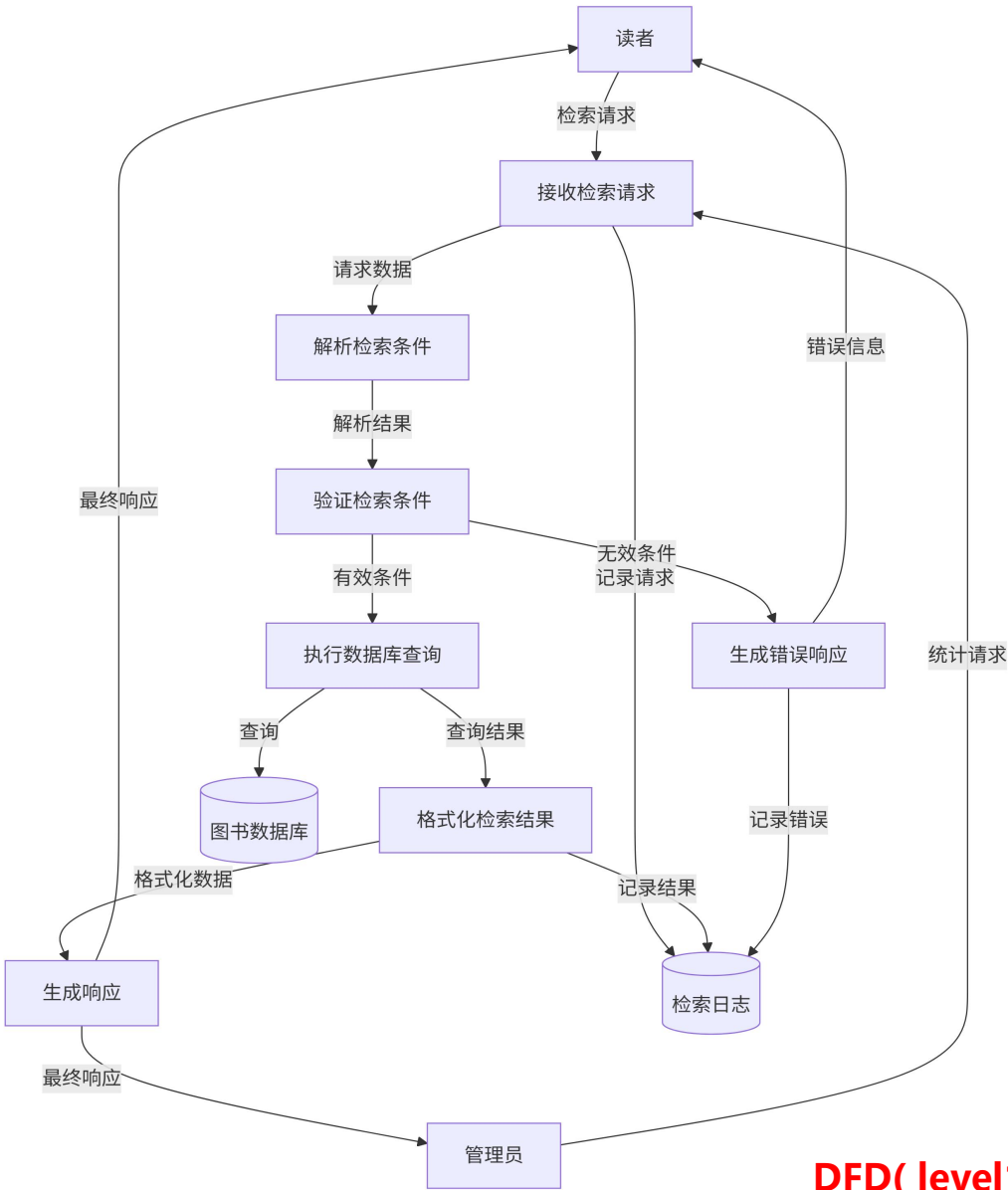


的类型
活动通路

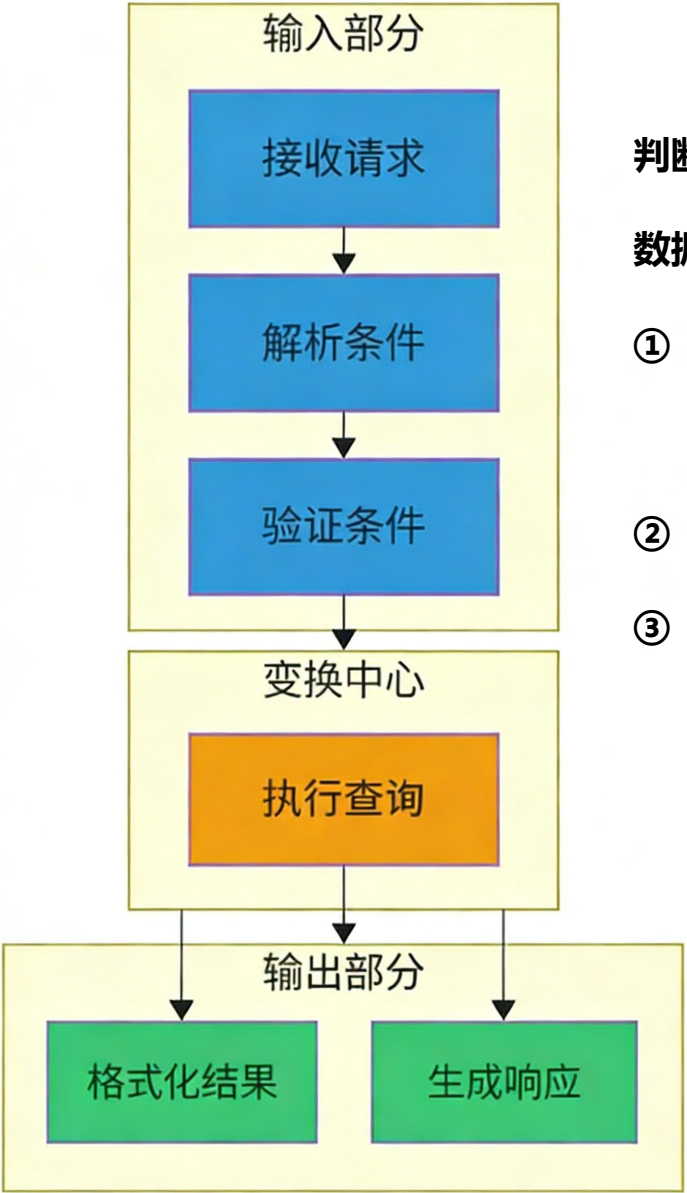
变换分析及其过程



设计步骤：第1步复查并精化数据流图（以图书馆图书查询系统为例）

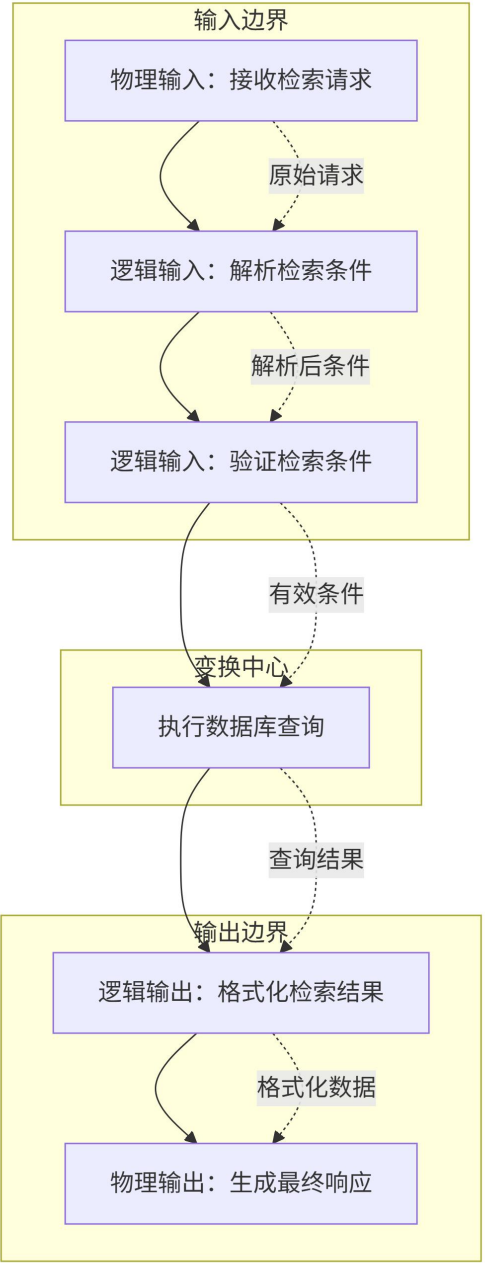


DFD(level1)



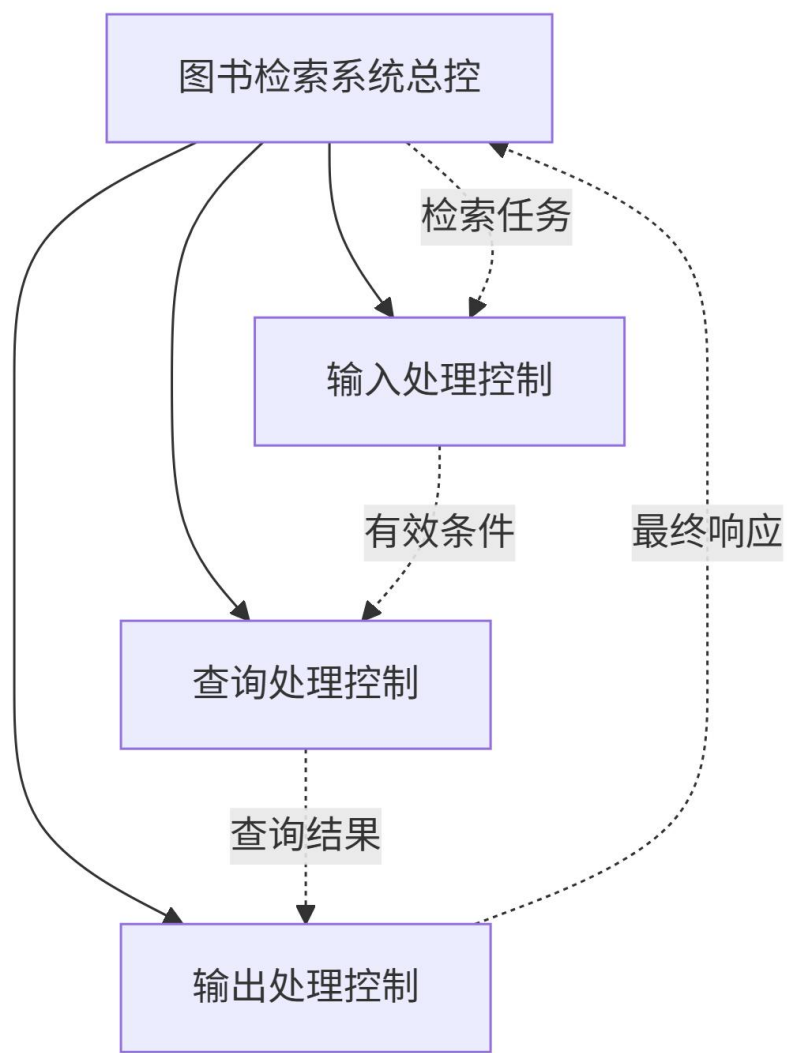
判断结果：这是一个变换型数据流。

- ① 有明显的输入路径 (接收→解析→验证)
- ② 核心变换 (数据库查询)
- ③ 明确的输出路径 (格式化→响应)

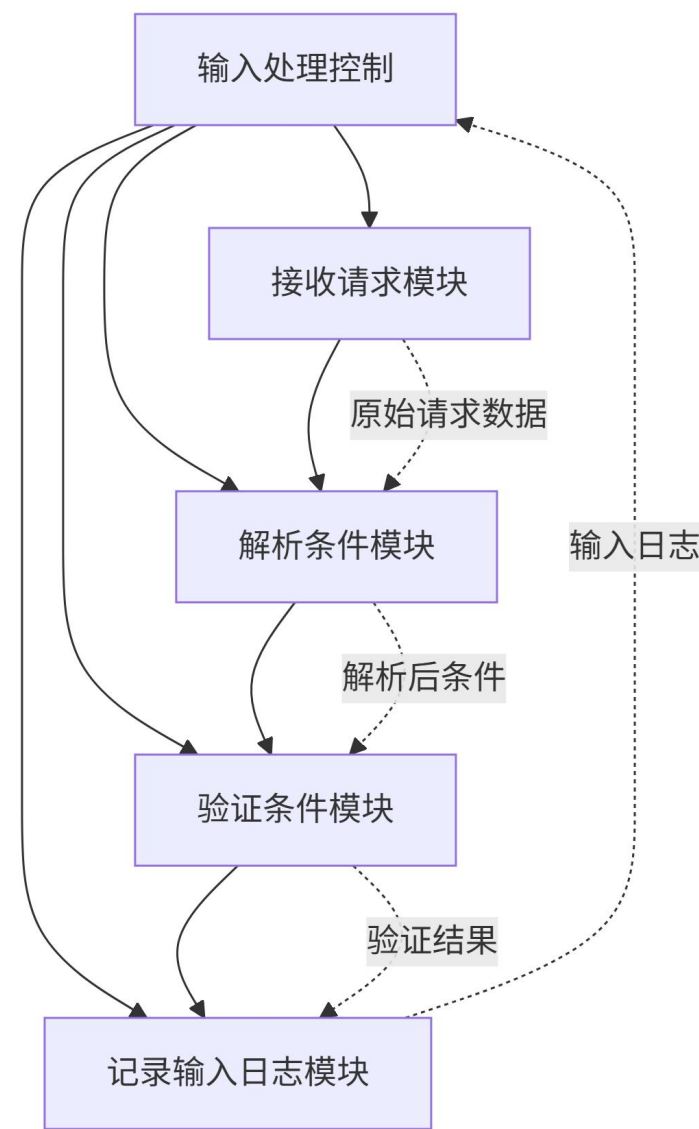


边界划分：

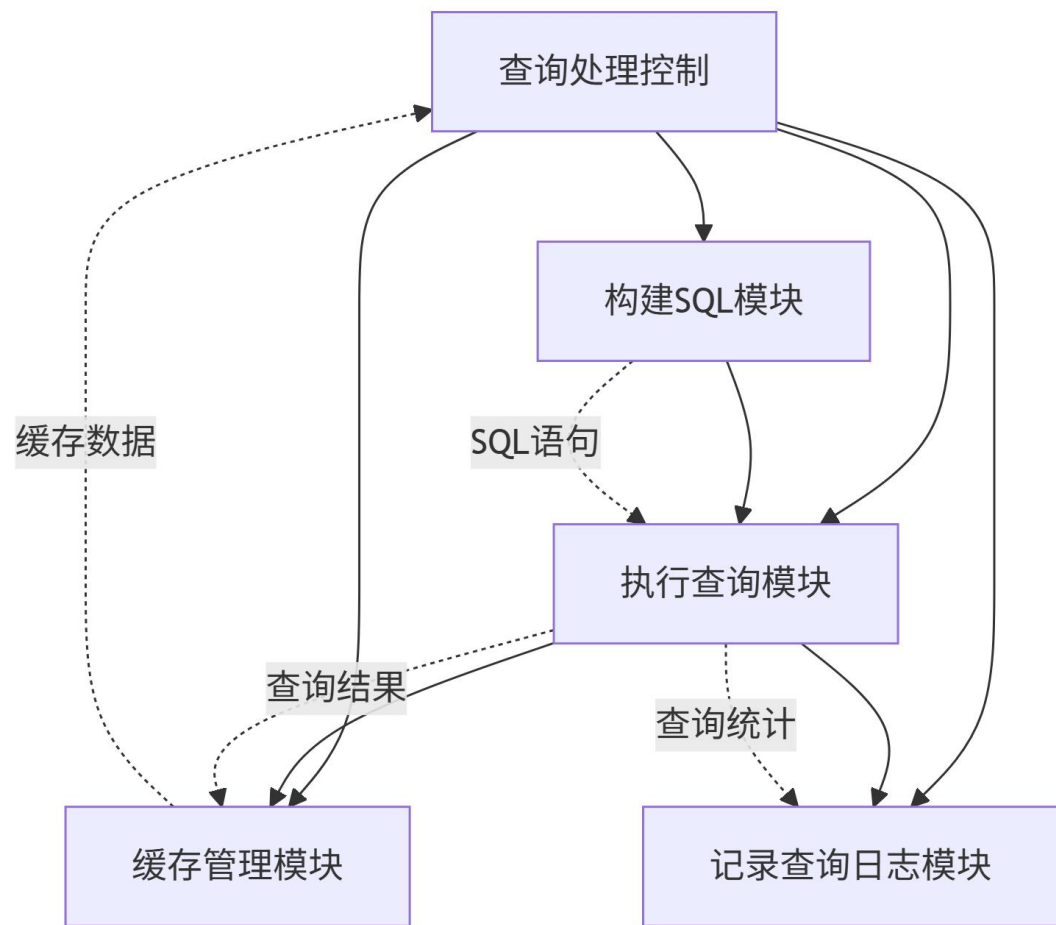
- ① 输入边界：从接收请求到验证完成
- ② 变换中心：数据库查询操作
- ③ 输出边界：从格式化结果到生成响应



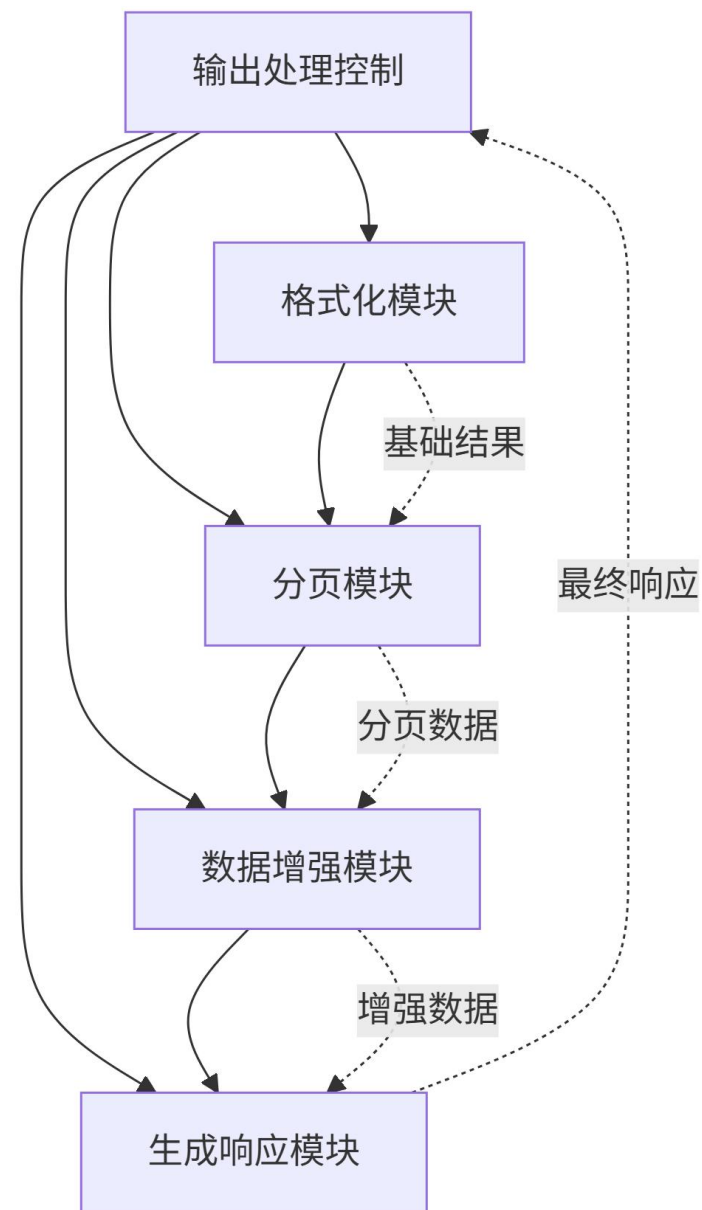
第一级分解的结构图



第二级：输入处理模块的分解

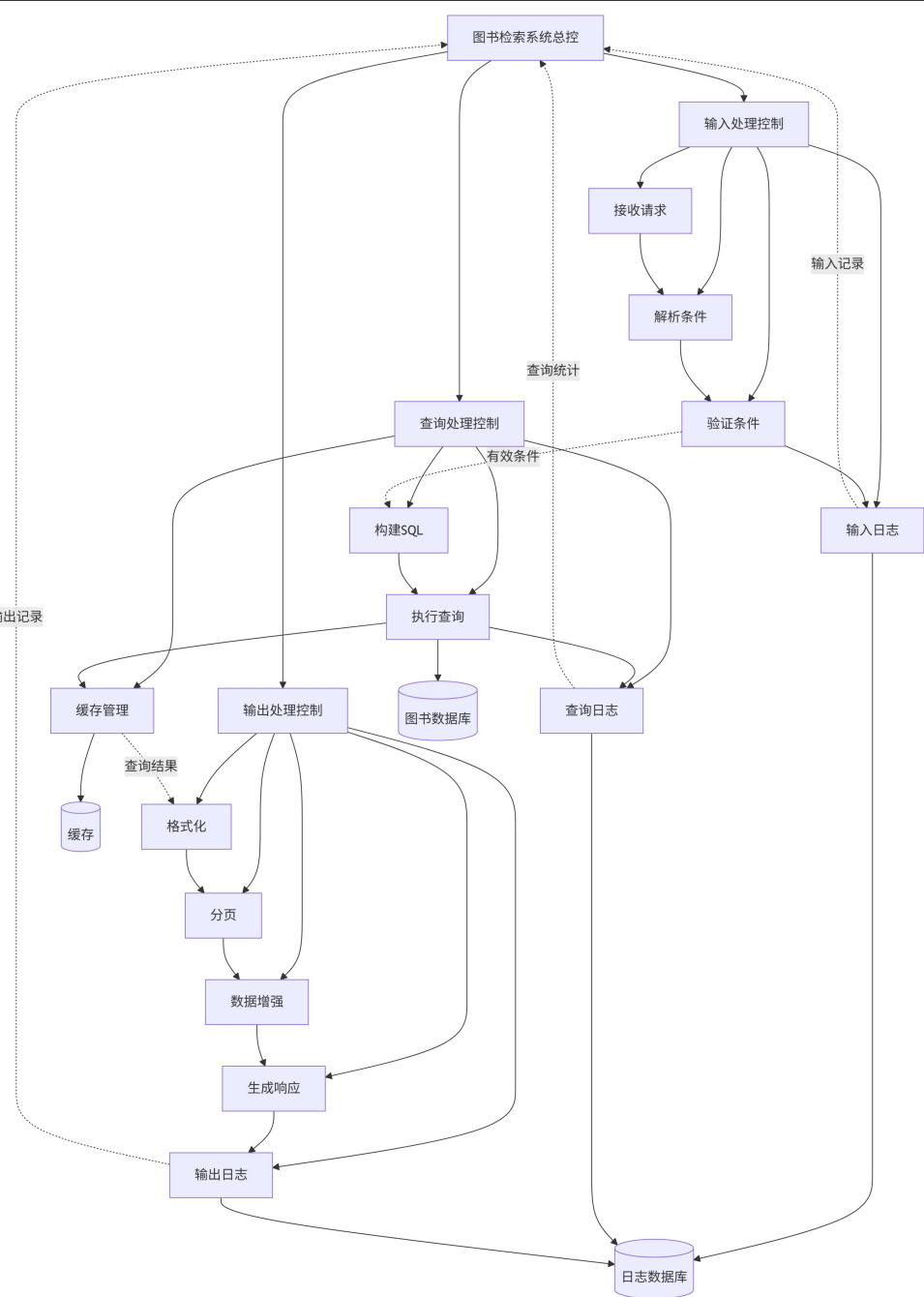


第二级：查询处理模块的分解



第二级：输出处理模块的分解

第6步：整合为完整结构图



第7步：定义模块接口

```
// 模块: ValidateCondition
// 输入: 解析后的检索条件
// 输出: 验证结果 (有效/无效 + 错误信息)
interface IConditionValidator {
    ValidationResult validate(SearchCondition condition);
}

class ValidationResult {
    boolean isValid;
    String errorMessage; // 如果无效
    SearchCondition validCondition; // 如果有效
}
```

验证条件模块接口

```
// 模块: BuildSQL
// 输入: 验证后的检索条件
// 输出: SQL 语句和参数
interface ISQLBuilder {
    SQLQuery buildQuery(SearchCondition condition);
}

class SQLQuery {
    String sql;
    List<Object> parameters;
    QueryType type; // SIMPLE/ADVANCED/FUZZY
}
```

构建SQL模块接口

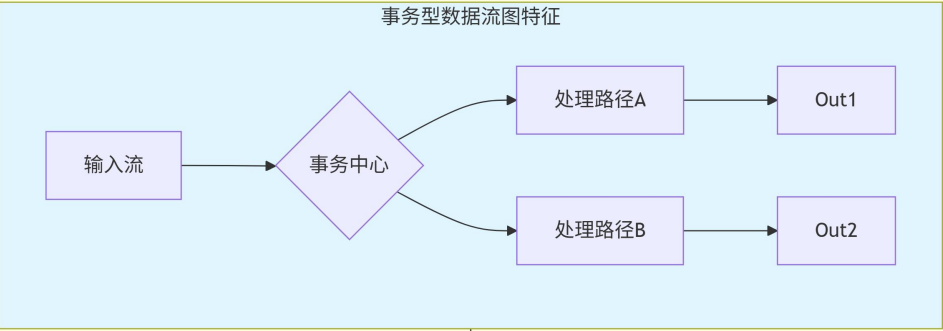
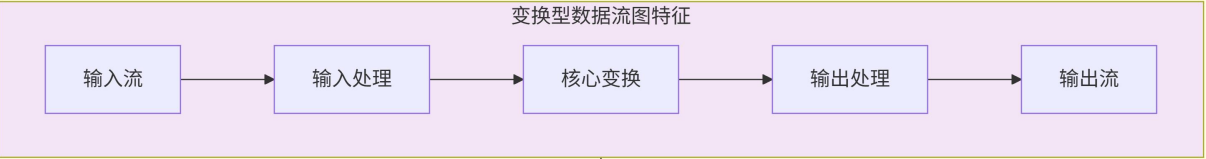
```
// 模块: FormatResult
// 输入: 原始查询结果
// 输出: 格式化的图书信息列表
interface IResultFormatter {
    List<FormattedBook> format(List<RawBookData> rawResults);
}

class FormattedBook {
    String title;
    String author;
    String publisher;
    int year;
    String status; // AVAILABLE/BORROWED
    String location;
}
```

格式化模块接口

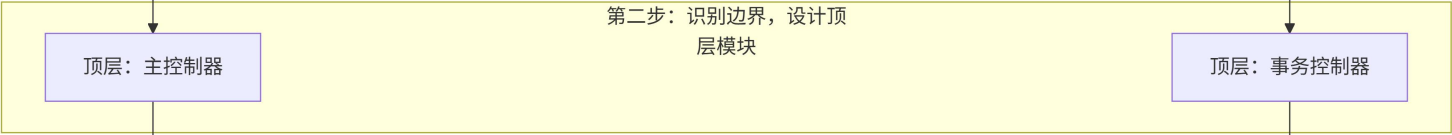
第8步：定义全局数据结构

- 1. SearchRequest (检索请求) :
 - requestId: string
 - searchType: enum [TITLE, AUTHOR, ISBN, CATEGORY]
 - keywords: list<string>
 - pageNumber: int
 - pageSize: int
 - timestamp: datetime
- 2. SearchCondition (检索条件) :
 - validated: boolean
 - conditions: list<ConditionItem>
 - pagination: PaginationInfo
- 3. ConditionItem:
 - field: string
 - operator: enum [EQUALS, CONTAINS, STARTS_WITH]
 - value: string
 - isRequired: boolean
- 4. QueryResult (查询结果) :
 - totalCount: int
 - books: list<BookRecord>
 - queryTime: float
 - fromCache: boolean
- 5. BookRecord:
 - id: string
 - title: string
 - author: string
 - isbn: string
 - category: string
 - status: enum [AVAILABLE, BORROWED, RESERVED]
 - location: string
 - publishYear: int
- 6. SearchResponse (检索响应) :
 - success: boolean
 - data: QueryResult
 - error: ErrorInfo (if not success)
 - requestId: string
 - timestamp: datetime



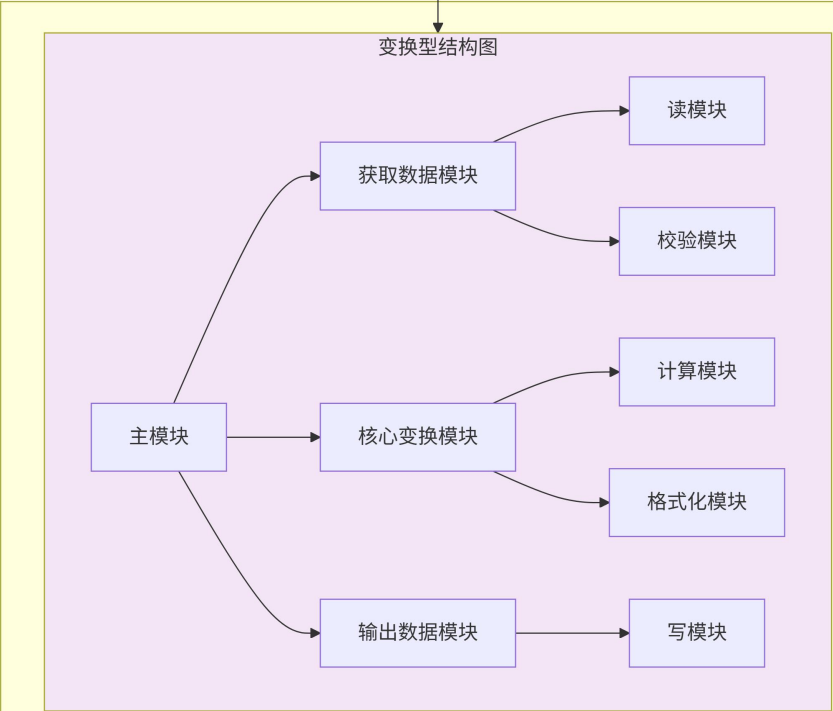
“识别输入、变换、输出流”

“识别事务中心与路径”

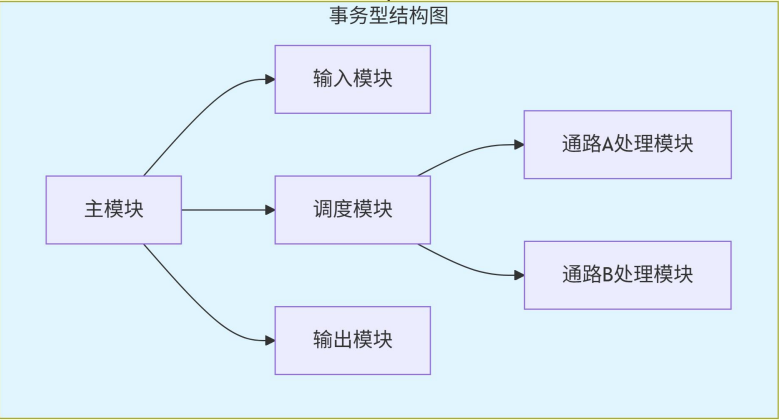


“以变换器为中心分解”

“以调度器为中心分解”



第三步：分解与映射，生成结构图



而且按

过程（算

的代码，以

 **THANKS** 